



Support de cours du module théorie des langages

Déterminisation : cas général
Minimisation des AEF

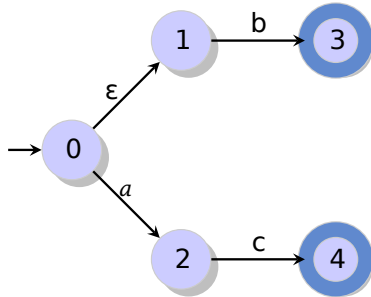
Présenté par Dr. T. Benouhiba





Problème

- On applique la détermination sur l'AEF suivant :



	a	b	c	ϵ
0	2	-	-	1
1	-	3	-	-
2	-	-	4	-
3	-	-	-	-
4	-	-	-	-

Détermination



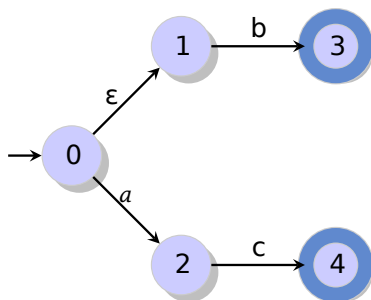
	a	b	c
0	2	-	-
2	-	-	4
4	-	-	-

Est-ce que le langage est le même ?



Comment faire ?

- La transition 0 vers 1 a été ignorée !
 - Solution : considérer que l'état 0 et 1 forme l'état initial



	a	b	c	ϵ
0	2	-	-	1
1	-	3	-	-
2	-	-	4	-
3	-	-	-	-
4	-	-	-	-

Déterminisation

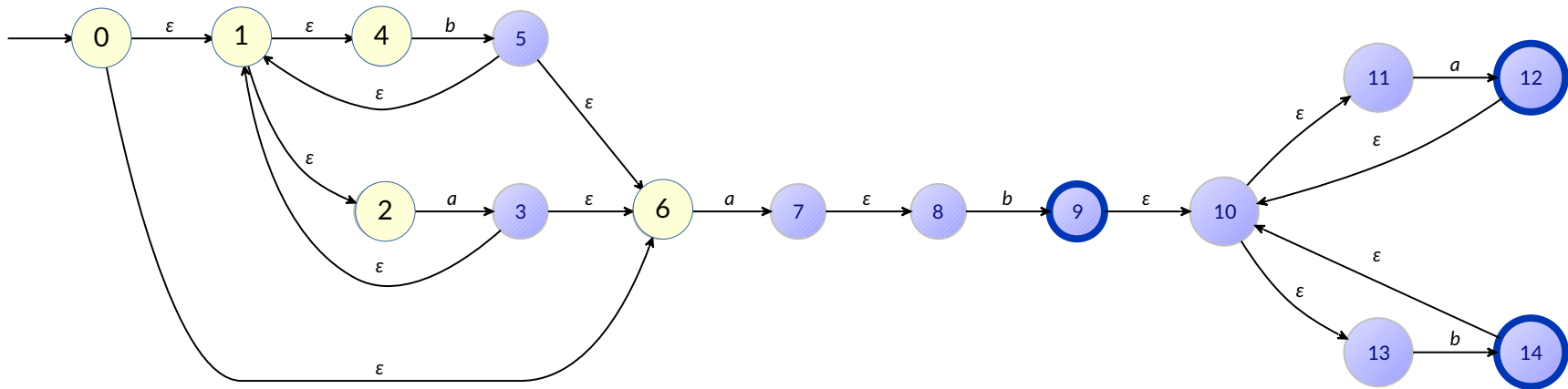


	a	b	c
0,1	2	3	-
2	-	-	4
4	-	-	-
3	-	-	-



L' ϵ -fermeture

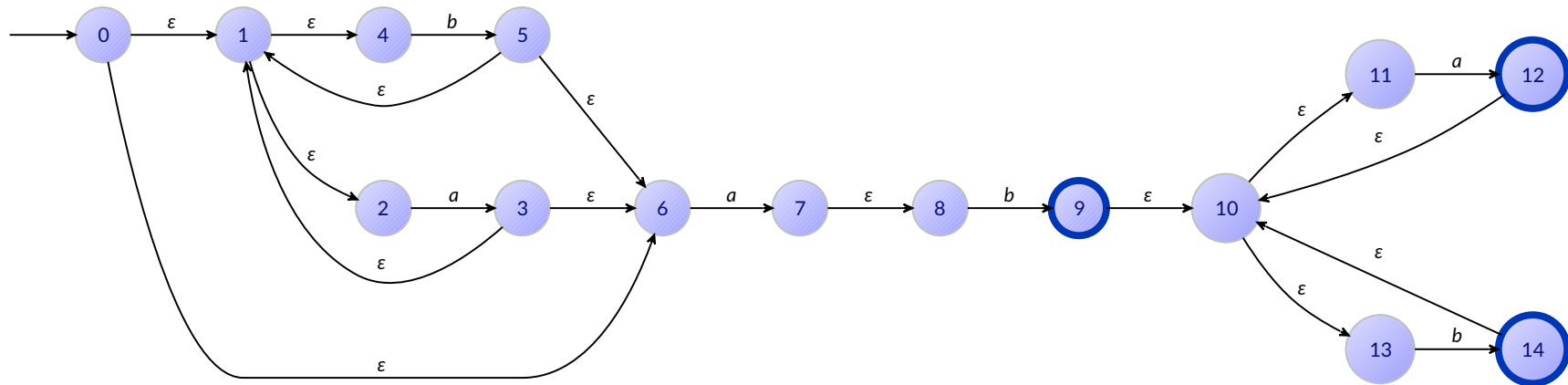
- $\epsilon_f(E)$ est l'ensemble des états accessibles depuis les états de E en ne passant que par des ϵ -transitions
 - Notons déjà que : $\epsilon_f(E) \supseteq E$
- Pour l'exemple suivant :
 - Calcul de : $\epsilon_f(\{0\}) = \{0, 1, 2, 4, 6\}$





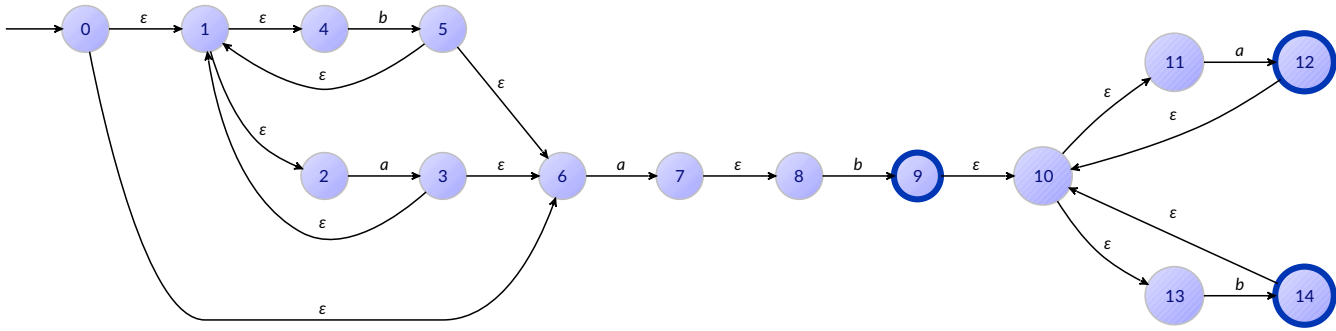
Algorithme de déterminisation

- Principe : on utilise le même algorithme que dans le cas simple, mais en appliquant tout le temps les ϵ -fermeture
- On souhaite déterminer l'automate suivant (acceptant tous les mots sur $\{a,b\}$ ayant le facteur **ab**)



Calcul

		a	b
0'	0, 1, 2, 4, 6	$\epsilon_f(\{3,7\})=\{1, 2, 3, 4, 6, 7, 8\}$	$\epsilon_f(\{5\})=\{1, 2, 4, 5, 6\}$
1'	1, 2, 3, 4, 6, 7, 8	$\epsilon_f(\{3,7\})=\{1, 2, 3, 4, 6, 7, 8\}$	$\epsilon_f(\{5,9\})=\{1, 2, 4, 5, 6, 9, 10, 11, 13\}$
2'	1, 2, 4, 5, 6	$\epsilon_f(\{3,7\})=\{1, 2, 3, 4, 6, 7, 8\}$	$\epsilon_f(\{5\})=\{1, 2, 4, 5, 6\}$
3'	1, 2, 4, 5, 6, 9, 10, 11, 13	$\epsilon_f(\{3,7,12\})=\{1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13\}$	$\epsilon_f(\{5,14\})=\{1, 2, 4, 5, 6, 10, 11, 13, 14\}$
4'	1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13	$\epsilon_f(\{3,7,12\})=\{1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13\}$	$\epsilon_f(\{5,9,14\})=\{1, 2, 4, 5, 6, 9, 10, 11, 13, 14\}$
5'	1, 2, 4, 5, 6, 10, 11, 13, 14	$\epsilon_f(\{3,7,12\})=\{1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13\}$	$\epsilon_f(\{5,14\})=\{1, 2, 4, 5, 6, 10, 11, 13, 14\}$
6'	1, 2, 4, 5, 6, 9, 10, 11, 13, 14	$\epsilon_f(\{3,7,12\})=\{1, 2, 3, 4, 6, 7, 8, 10, 11, 12, 13\}$	$\epsilon_f(\{5,14\})=\{1, 2, 4, 5, 6, 10, 11, 13, 14\}$





Renommage et paramètres

- État initial : 0'
- États finaux : {3',4',5',6'}

	a	b
0'	1'	2'
1'	1'	3'
2'	1'	2'
3'	4'	5'
4'	4'	6'
5'	4'	5'
6'	4'	5'



Terminaison et nombre d'états

- L'algorithme de détermination procède comme suit :
 - Tant qu'il y a de nouveaux états dans la table, calculer la fonction de transition
 - L'algorithme se termine-t-il toujours ? Pourquoi ?
 - Réponse : oui
 - Nombre maximal d'états pour un AEF ayant p états :
 - Chaque nouvel état est un sous-ensemble des états de l'AEF non-déterministe
 - Le nombre de sous-ensembles d'un ensemble à p éléments est 2^p
 - Le nombre maximal des états après détermination est $2^p - 1$
 - Conclusion : l'algorithme de détermination se termine toujours (mais peut produire beaucoup d'états)



Troisième partie : minimisation des AEF



Objectifs de la détermination

- La détermination des AEF vise à créer des automates plus efficaces en terme de temps d'analyse
 - Rappel : pour analyse un mot w de longueur n ($|w|=n$), on a :
 - Pour un AEF déterministe, le nombre de configurations est $f(n)=n+1$
 - il est donc indépendant du nombre d'états ou de la topologie de l'AEF
 - Pour un AEF non-déterministe, le nombre de configurations peut même être $f(n)=a^n$
 - il peut y avoir une dépendance vis-à-vis du nombre d'états et/ou de la topologie de l'AEF





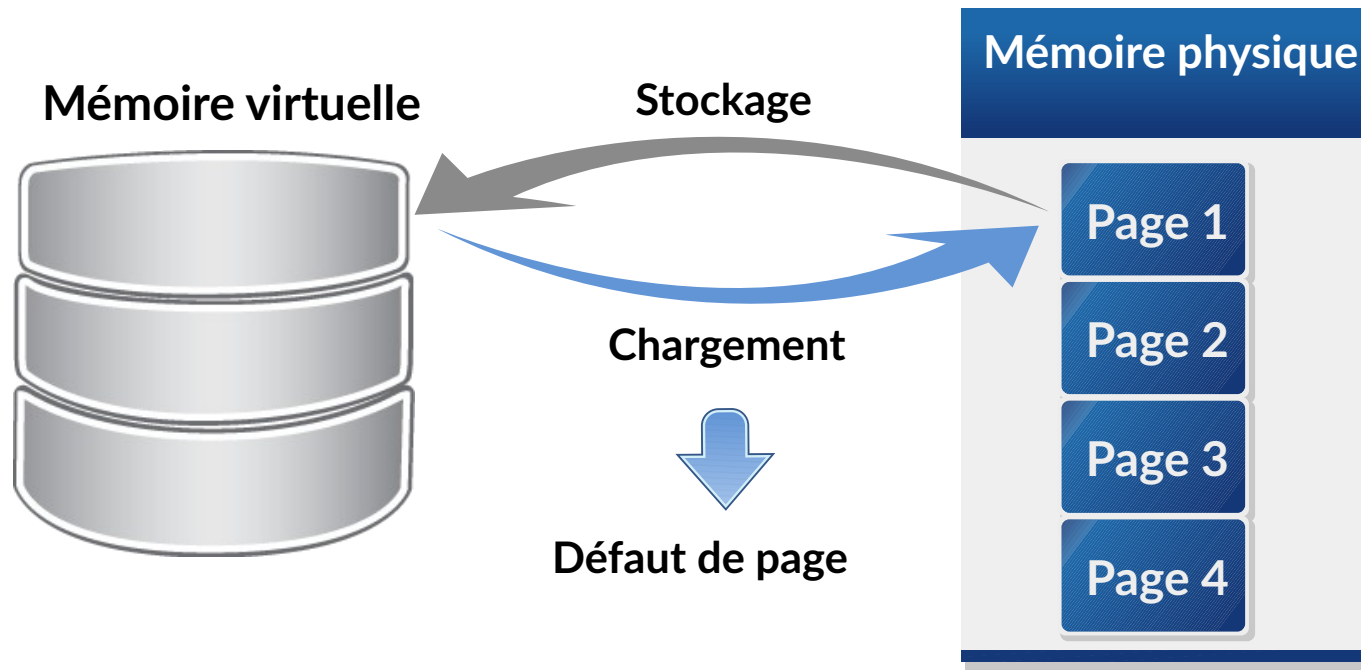
Effet de la détermination

- Pour un AEF non-déterministe à p états, la procédure de détermination peut produire jusqu'à $2^p - 1$ états !
 - Exemple : pour certains AEF ayant 100 états, on peut obtenir après détermination $2^{100} - 1$ états (de l'ordre de 10^{30})
- Mais, en théorie un AEF à 100 états et un AEF à 10^6 états (acceptant le même langage) prennent le même temps pour analyser n'importe quel mot
 - En pratique, est-ce pareil ?
- Un AEF sera transformé en un programme
 - Plus il y a des états dans l'AEF, plus le programme contiendra des instructions
 - → Un AEF ayant peu d'états occupera moins d'espace (et il sera plus facile à maintenir aussi)



Aspects pratiques

- Dans les systèmes d'exploitation, la mémoire utilisée par les processus est plus grande que celle de la mémoire physique (utilisation de la mémoire virtuelle)





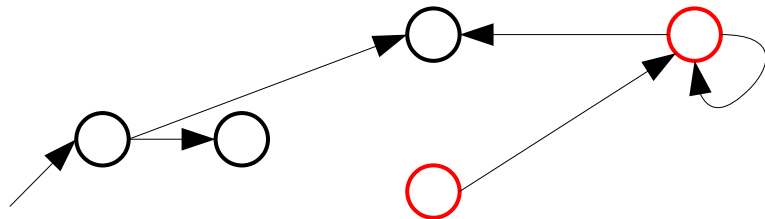
Intérêt et principes de la minimisation

- En pratique, le temps d'analyse peut dépendre du nombre d'états, on a donc intérêt à réduire le nombre des états
 - On appelle minimisation, d'un AEF déterministe, la construction d'un AEF comportant le minimum d'états possibles pour accepter le même langage
- Principes :
 - Nettoyer l'AEF : certains états peuvent ne jamais être utilisés, autant les supprimer
 - Détecter les états ayant la même fonction



Élimination des états inutiles (inaccessibles)

- Certains états peuvent ne jamais être visités : par les mots acceptés ou les mots rejetés
 - Ils ne servent à rien, on peut les supprimer
 - On les appelle des états inaccessibles
- Une autre définition venant de la théorie des graphes :
 - un état est dit inaccessible s'il n'existe pas de chemin entre l'état initial et cet état





Détection des états inaccessibles

- On applique un algorithme de marquage (en profondeur, en largeur ou autre)
- On considère l'AEF suivant (l'état initial est 1) :
 - L'état 7 n'est pas marqué, il est donc inaccessible
 - On peut enlever l'état 7 sans changer de langage

État	a	b
● 1	2	5
● 2	2	4
● 3	3	2
● 4	5	3
● 5	4	6
● 6	6	1
7	5	7



Remarques

- L'élimination des états inaccessibles permet juste de nettoyer l'AEF, elle ne permet pas de réduire énormément le nombre des états
- Si on applique cette procédure à un AEF obtenu par déterminisation, alors tous les états seront accessibles



Réduction du nombre d'états

- Principe :
 - Soit l'AEF suivant (initial=0, finaux={2,3})
 - L'AEF se comporte de la même façon s'il est dans l'état 2 ou 3
 - Les états 2 et 3 ont la même fonction $\rightarrow$ on peut alors les fusionner
 - Le nombre d'états passe à 3 seulement

État	a	b
0	1	0
1	1	2
2	3	2
3	3	2



État	a	b
0	1	0
1	1	2
2	2	2



Réduction du nombre d'états

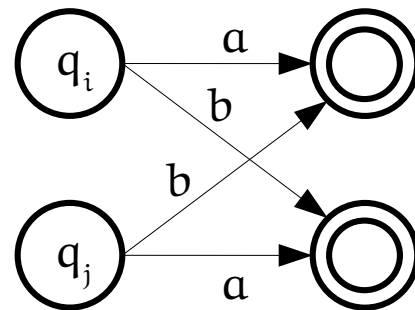
- Et avec cet AEF (initial=1, finaux={1,2}) ?
 - On ne trouve pas d'états ayant la même fonction !
 - On a besoin d'une meilleure définition

État	a	b
1	2	5
2	2	4
3	3	2
4	5	3
5	4	6
6	6	1



Réduction du nombre d'états

- On considère la situation :
 - Pour atteindre un état final à partir de q_i , on utilise les mots a et b
 - Pour atteindre un état final à partir de q_j , on utilise les mots a et b
 - On dit que les états q_i et q_j sont β -équivalents (ils utilisent les mêmes mots pour atteindre les états finaux)
 - $q_i \beta q_j$
- La relation β est une relation d'équivalence
 - Chaque classe d'équivalence représente un état de l'AEF minimal





Regroupement des états

- Un algorithme permet de regrouper les états qui sont équivalents :
- Étape 1 :
 - Créer deux classes d'états :
 - $A=\{1,2\}$ (mettre tous les états finaux)
 - $B=\{3,4,5,6\}$ (mettre tous les états non-finaux)
 - On part de l'hypothèse que l'AEF minimal contient seulement deux états
 - Un état final ne doit jamais être mis avec un état non-final...
- Étape 2 :
 - Vérifier la cohérence des classes

État	a	b
1	2	5
2	2	4
3	3	2
4	5	3
5	4	6
6	6	1



Cohérence d'une classe

- Une classe **X** est cohérente par rapport à un symbole **a** si :
 - Toutes les transitions des états de **X** avec le symbole **a** mènent à la même classe
- Une classe est cohérente si elle est cohérente par rapport à tous les symboles de l'alphabet
- Si une classe n'est pas cohérente, il faut l'éclater (la diviser en deux ou plusieurs classes)



Suite de l'algorithme

- On procède alors par itération, à chaque itération on vérifie la cohérence de chaque classe
- 1^{ère} itération ($A=\{1,2\}$, $B=\{3,4,5,6\}$) :

A	a	b
1	$2 \in A$	$5 \in B$
2	$2 \in A$	$4 \in B$

A est
cohérente

B	a	b
3	$3 \in B$	$2 \in A$
4	$5 \in B$	$3 \in B$
5	$4 \in B$	$6 \in B$
6	$6 \in B$	$1 \in A$

B n'est pas cohérente
Elle est éclatée en $B=\{3,6\}$ et $C=\{4,5\}$



Suite de l'algorithme

- A chaque itération, si au moins une classe est modifiée, alors on refait une nouvelle itération
- 2^{ème} itération ($A=\{1,2\}$, $B=\{3,6\}$, $C=\{4,5\}$) :

A	a	b
1	$2 \in A$	$5 \in C$
2	$2 \in A$	$4 \in C$

✓ ✓

A est cohérente

B	a	b
3	$3 \in B$	$2 \in A$
6	$6 \in B$	$1 \in A$

✓ ✓

B est cohérente

C	a	b
4	$5 \in C$	$3 \in B$
5	$4 \in C$	$6 \in B$

✓ ✓

C est cohérente

- Aucune modification à la deuxième itération, donc on arrête l'algorithme



Résultats de l'algorithme

A	a	b
1	$2 \in A$	$5 \in C$
2	$2 \in A$	$4 \in C$

B	a	b
3	$3 \in B$	$2 \in A$
6	$6 \in B$	$1 \in A$

C	a	b
4	$5 \in C$	$3 \in B$
5	$4 \in C$	$6 \in B$

- L'AEF contient trois états : A, B et C

État	a	b
A	A	C
B	B	A
C	C	B

- État initial = la classe qui contient l'ancien état initial : A
- États finaux = les classes qui contiennent des états finaux : {A}



Remarques

- L'automate obtenu est minimal et accepte le même langage
- L'application de la minimisation à un automate minimisé ne permet de rien gagner
- On peut caractériser alors un langage régulier par son AEF déterministe et minimal (qui est donc **unique**)
- Application : comment peut-on décider si deux langages réguliers **L** et **L'** sont égaux ?